

Họ và tên:

Mã số Sinh viên:

Lớp:

HỆ ĐIỀU HÀNH BÁO CÁO LAB 5

TÓM TẮT BÁO CÁO

1. Bài tập thực hành

	5.5.1	5.5.2	5.5.3	5.5.4
Phương pháp	☐	☐	☐	☐
Minh chứng	☐	☐	☐	☐
Giải thích	☐	☐	☐	☐

2. Bài tập ôn tập

	5.6.1
Phương pháp	☐
Minh chứng	☐
Giải thích	☐

Tự đánh giá:

**Lưu ý: Xuất file PDF, đặt tên theo cú pháp:*

<MSSV>_LABx.pdf

BÁO CÁO

I. Bài tập thực hành

5.5.1. Hiện thực hóa mô hình trong ví dụ 5.3.1.2, tuy nhiên thay bằng điều kiện sau:
 $\text{sells} \leq \text{products} \leq \text{sells} + [4 \text{ số cuối của MSSV}]$

- **Phương pháp:** tạo 2 semaphore semA và semB, và 2 biến đếm products và sells (ban đầu bằng 0). semA và semB ràng buộc điều kiện ở trên. Ở mỗi vòng lặp, luồng A chờ semA (được luồng B nhả khi có sản phẩm) rồi tăng sells và nhả semB, luồng B chờ semB (có giới hạn 346), tăng products rồi nhả semA.

```

34 #include <pthread.h>
33 #include <semaphore.h>
32 #include <stdio.h>
31
30 sem_t semA, semB;
29 int products = 0, sells = 0;
28
27 void *
26 processA(void *ignore)
25 {
24 >     while (1) {
23 >         sem_wait(&semA);
22 >         printf("A: sell, sells = %d\n", ++sells);
21 >         sem_post(&semB);
20 >     }
19 }
18
17 void *
16 processB(void *ignore)
15 {
14 >     while (1) {
13 >         sem_wait(&semB);
12 >         printf("B: produce, products = %d\n", ++products);
11 >         sem_post(&semA);
10 >     }
9 }

```

```
6 int
7 main(void)
8 {
9     pthread_t threadA, threadB;
10    sem_init(&semA, 0, 0);
11    sem_init(&semB, 0, 346);
12    pthread_create(&threadA, NULL, &processA, NULL);
13    pthread_create(&threadB, NULL, &processB, NULL);
14    pthread_join(threadA, NULL);
15    pthread_join(threadB, NULL);
16 }
```

- Kết quả chạy (trích):

```
B: produce, products = 823
B: produce, products = 824
B: produce, products = 825
A: sell, sells = 480
A: sell, sells = 481
A: sell, sells = 482
A: sell, sells = 483
B: produce, products = 826
B: produce, products = 827
B: produce, products = 828
B: produce, products = 829
A: sell, sells = 484
A: sell, sells = 485
A: sell, sells = 486
A: sell, sells = 487
B: produce, products = 830
:
```

- **Giải thích:** ở hàm main(), khởi tạo 2 semaphore semA = 0, semB = 346 (0346 là 4 số cuối của mã số sinh viên), khởi tạo 2 luồng threadA, threadB chạy lần lượt processA() và processB().

+ Ở processA(), sem_wait(&semA) kiểm tra nếu semA = 0 (tức không có hàng) thì chặn luồng, còn không giảm semA đi 1 giá trị, tăng sells thêm 1 rồi in ra, sem_post(&semB) tăng semB thêm 1 giá trị.

+ Ở processB(), sem_wait(&semB) kiểm tra nếu semB = 0 thì chặn luồng, còn không giảm semB đi 1 giá trị, tăng products thêm 1 rồi in giá trị mới, sem_post(&semA) tăng semA thêm 1 giá trị.

+ Hai lệnh `pthread_join()` trong hàm `main()` đảm bảo hàm `main()` không thoát trước khi hai luồng thực thi xong.

5.5.2. Cho một mảng `a` được khai báo như một mảng số nguyên có thể chứa `n` phần tử, `a` được khai báo như một biến toàn cục. Viết chương trình bao gồm 2 thread chạy song song:

- Một thread làm nhiệm vụ sinh ra một số nguyên ngẫu nhiên sau đó bỏ vào `a`. Sau đó đếm và xuất ra số phần tử của `a` có được ngay sau khi thêm vào.

- Thread còn lại lấy ra một phần tử trong `a` (phần tử bất kỳ, phụ thuộc vào người lập trình). Sau đó đếm và xuất ra số phần tử của `a` có được ngay sau khi lấy ra, nếu không có phần tử nào trong `a` thì xuất ra màn hình “Nothing in array `a`”.

a. Chạy thử và tìm ra lỗi khi chạy chương trình trên khi chưa được đồng bộ.

- **Phương pháp:** tạo 2 luồng, mỗi luồng là 1 vòng lặp. Luồng A lấy ngẫu nhiên 1 số, thêm vào mảng rồi in số phần tử, luồng B lấy ngẫu nhiên 1 vị trí đã thêm phần tử, xóa rồi in số phần tử mới.

```
23 #include <pthread.h>
22 #include <stdio.h>
21 #include <stdlib.h>
20
19 int *a, n;
18 int count = 0;
17
16 void *
15 processA(void *ignore)
14 {
13 >     while (1) {
12 >         >     if (count < n) {
11 >             >         a[count++] = arc4random_uniform(1000);
10 >             >         printf("A: insert, count = %d\n", count);
9 >             >     } else
8 >             >         printf("A: full, count = %d\n", count);
7 >         }
6 }
5
```

```

3
4 void *
5 processB(void *ignore)
6 {
7     while (1) {
8         if (count <= 0)
9             printf("Nothing in array a.\n");
10        else {
11            int randpos = arc4random_uniform(count);
12            for (int i = randpos; i < count - 1; i++)
13                a[i] = a[i + 1];
14            a[--count] = 0;
15            printf("B: delete a[%d], count = %d\n", randpos, count);
16        }
17    }
18 }
19
20 int
21 main(int argc, char *argv[])
22 {
23     pthread_t threadA, threadB;
24     n = atoi(argv[1]);
25     a = malloc(n * sizeof(int));
26     pthread_create(&threadA, NULL, &processA, NULL);
27     pthread_create(&threadB, NULL, &processB, NULL);
28     pthread_join(threadA, NULL);
29     pthread_join(threadB, NULL);
30 }
~

```

- Kết quả chạy với n = 10 (trích):

```

B: delete a[3], count = 4
A: insert, count = 6
A: insert, count = 4
A: insert, count = 5
A: insert, count = 6
B: delete a[2], count = 3
B: delete a[6], count = 6
B: delete a[2], count = 5
B: delete a[2], count = 4
A: insert, count = 7
A: insert, count = 4
A: insert, count = 5
B: delete a[2], count = 3
B: delete a[3], count = 5
B: delete a[0], count = 4
:

```

- **Lỗi:** chương trình không được đồng bộ nên 2 luồng thực thi không đồng đều, dẫn tới sai lệch về giá trị của biến count và mảng a.

b. Thực hiện đồng bộ hóa với semaphore.

- **Phương pháp:** khai báo ba semaphore semA (mặc định là số phần tử), semB (mặc định là 0), semC (mặc định là 1). Ở mỗi vòng lặp, luồng A chờ semA và semC, sau khi in xong thì nhả semB và semC, luồng B chờ semB và semC, sau khi in xong thì nhả semA và semC.

```

7 #include <pthread.h>
6 #include <semaphore.h>
5 #include <stdio.h>
4 #include <stdlib.h>
3
2 sem_t semA, semB, semC;
1 int *a, count = 0, n;
8
1 void *
2 processA(void *ignore)
3 {
4     while (1) {
5         sem_wait(&semA);
6         sem_wait(&semC);
7         a[count++] = arc4random_uniform(1000);
8         printf("A: insert, count = %d\n", count);
9         sem_post(&semC);
10        sem_post(&semB);
11    }
12 }
13
14 void *
15 processB(void *ignore)
16 {
17     while (1) {
18         sem_wait(&semB);
19         sem_wait(&semC);
20         int randpos = arc4random_uniform(count);
21         for (int i = randpos; i < count - 1; i++)
22             a[i] = a[i + 1];
23         a[--count] = 0;
24         printf("B: delete a[%d], count = %d\n", randpos, count);
25         sem_post(&semC);
26         sem_post(&semA);
27     }
28 }
29

```

```
10 int
11 main(int argc, char *argv[])
12 {
13     pthread_t threadA, threadB;
14     n = atoi(argv[1]);
15     a = malloc(n * sizeof(int));
16     sem_init(&semA, 0, n);
17     sem_init(&semB, 0, 0);
18     sem_init(&semC, 0, 1);
19     pthread_create(&threadA, NULL, &processA, NULL);
20     pthread_create(&threadB, NULL, &processB, NULL);
21     pthread_join(threadA, NULL);
22     pthread_join(threadB, NULL);
23 }
```

- **Kết quả chạy với n = 10 (trích):**

```
A: insert, count = 4
A: insert, count = 5
B: delete a[1], count = 4
B: delete a[0], count = 3
B: delete a[1], count = 2
B: delete a[1], count = 1
B: delete a[0], count = 0
A: insert, count = 1
A: insert, count = 2
A: insert, count = 3
A: insert, count = 4
A: insert, count = 5
A: insert, count = 6
A: insert, count = 7
A: insert, count = 8
A: insert, count = 9
A: insert, count = 10
B: delete a[7], count = 9
B: delete a[8], count = 8
B: delete a[0], count = 7
B: delete a[2], count = 6
A: insert, count = 7
A: insert, count = 8
```

- **Giải thích:** ở hàm main(), khởi tạo ba semaphore: semA đếm số phần tử chưa nhập (mặc định là n), semB đếm số phần tử đã nhập (mặc định là 0), semC (mặc định là 1) có tác dụng như khóa mutex. Khởi tạo hai luồng threadA, threadB chạy hai hàm processA(), processB() tương ứng.

+ Ở cả 2 hàm processA(), processB(), sem_wait(&semC) kiểm tra xem có luồng nào khác đang truy cập vào *a* và *count* không (khi đó semC = 0), nếu có thì chặn luồng, nếu không thì semC = 0 để khóa lại, luồng hiện tại thao tác lên *a* và *count*, sem_post(&semC) tăng semC lên 1 để luồng khác có thể truy cập.

+ Ở process A, sem_wait(&semA) kiểm tra nếu semA = 0 (tức mảng *a* đã đầy) thì chặn luồng, còn không thì chờ semC. Khi semC được nhả, chọn một số ngẫu nhiên trong đoạn [0, 999] thêm vào mảng *a*, tăng biến đếm count lên 1, in số phần tử mới rồi nhả semC và tăng semB lên 1.

+ Ở process B, sem_wait(&semB) kiểm tra nếu semB = 0 (tức mảng *a* chưa có phần tử nào) thì chặn luồng, còn không thì chờ semC. Khi semC được nhả, chọn một phần tử ngẫu nhiên trong *a* rồi xóa nó, giảm biến đếm count xuống 1, in số phần tử mới rồi nhả semC và tăng semA lên 1.

5.5.3. Cho 2 process A và B chạy song song như sau:

int x = 0;	
PROCESS A	PROCESS B
<pre> processA() { while(1){ x = x + 1; if (x == 20) x = 0; print(x); } } </pre>	<pre> processB() { while(1){ x = x + 1; if (x == 20) x = 0; print(x); } } </pre>

Hiện thực mô hình trên C trong hệ điều hành Linux và nhận xét kết quả.

- **Phương pháp:** dùng `pthread_create()`.

```
34 #include <pthread.h>
33 #include <stdio.h>
32
31 int x = 0;
30
29 void *
28 processA(void *ignore)
27 {
26 >     while (1) {
25 >         >         x = x + 1;
24 >         >         if (x == 20)
23 >         >             x = 0;
22 >         >         printf("A: x = %d\n", x);
21 >     }
20 }
19
18 void *
17 processB(void *ignore)
16 {
15 >     while (1) {
14 >         >         x = x + 1;
13 >         >         if (x == 20)
12 >         >             x = 0;
11 >         >         printf("B: x = %d\n", x);
10 >     }
9 }
8
7 int
6 main(void)
5 {
4 >     pthread_t threadA, threadB;
3 >     pthread_create(&threadA, NULL, &processA, NULL);
2 >     pthread_create(&threadB, NULL, &processB, NULL);
1 >     pthread_join(threadA, NULL);
35 >     pthread_join(threadB, NULL);
1 }

~
normal 553.c [c] utf-8 [unix] 97% 35:21
"553.c" 36L, 495B written
```

- Kết quả chạy:

```
A: x = 1
A: x = 3
A: x = 4
A: x = 5
A: x = 6
A: x = 7
A: x = 8
A: x = 9
A: x = 10
A: x = 11
A: x = 12
A: x = 13
A: x = 14
A: x = 15
A: x = 16
A: x = 17
A: x = 18
A: x = 19
B: x = 2
B: x = 1
B: x = 2
B: x = 3
B: x = 4
B: x = 5
B: x = 6
B: x = 7
B: x = 8
B: x = 9
B: x = 10
B: x = 11
B: x = 12
B: x = 13
B: x = 14
B: x = 15
B: x = 16
B: x = 17
B: x = 18
B: x = 19
:
```

- **Giải thích:** chương trình có lỗi vùng tranh chấp. Thay vì x chạy từ 0-20 thì xuất hiện dòng "B: x = 2" cho thấy có lúc cả processA() lẫn processB() đều thực thi vùng tranh chấp, do đó gây xung đột và kết quả sai.

5.5.4. Đồng bộ với mutex để sửa lỗi bất hợp lý trong kết quả của mô hình Bài 3.

- **Phương pháp:** khởi một mutex, ở mỗi hàm, khóa mutex lại trước khi tiến vào vùng tranh chấp, mở khóa khi thoát khỏi vùng tranh chấp.

```

34 #include <pthread.h>
33 #include <stdio.h>
32
31 pthread_mutex_t mutex;
30 int x = 0;
29
28 void *
27 processA(void *ignore)
26 {
25 >     while (1) {
24 >         pthread_mutex_lock(&mutex);
23 >         x = x + 1;
22 >         if (x == 20)
21 >             x = 0;
20 >         printf("A: x = %d\n", x);
19 >         pthread_mutex_unlock(&mutex);
18 >     }
17 }
16
15 void *
14 processB(void *ignore)
13 {
12 >     while (1) {
11 >         pthread_mutex_lock(&mutex);
10 >         x = x + 1;
9 >         if (x == 20)
8 >             x = 0;
7 >         printf("B: x = %d\n", x);
6 >         pthread_mutex_unlock(&mutex);
5 >     }
4 }
3

```

```
2 int
1 main(void)
35 {
1 >     pthread_t threadA, threadB;
2 >     pthread_mutex_init(&mutex, NULL);
3 >     pthread_create(&threadA, NULL, &processA, NULL);
4 >     pthread_create(&threadB, NULL, &processB, NULL);
5 >     pthread_join(threadA, NULL);
6 >     pthread_join(threadB, NULL);
7 }
```

- Kết quả chạy:

```
A: x = 0
A: x = 1
A: x = 2
A: x = 3
A: x = 4
A: x = 5
A: x = 6
A: x = 7
A: x = 8
A: x = 9
A: x = 10
A: x = 11
A: x = 12
A: x = 13
A: x = 14
A: x = 15
A: x = 16
A: x = 17
A: x = 18
A: x = 19
A: x = 0
B: x = 1
B: x = 2
B: x = 3
B: x = 4
B: x = 5
B: x = 6
B: x = 7
B: x = 8
B: x = 9
B: x = 10
```

```
B: x = 11
B: x = 12
B: x = 13
B: x = 14
B: x = 15
B: x = 16
B: x = 17
B: x = 18
B: x = 19
B: x = 0
B: x = 1
B: x = 2
B: x = 3
B: x = 4
B: x = 5
B: x = 6
B: x = 7
A: x = 8
A: x = 9
A: x = 10
A: x = 11
A: x = 12
A: x = 13
A: x = 14
A: x = 15
A: x = 16
A: x = 17
A: x = 18
A: x = 19
A: x = 0
A: x = 1
A: x = 2
A: x = 3
A: x = 4
A: x = 5
A: x = 6
:
```

- **Giải thích kết quả:** ở hàm main(), khởi tạo khóa *mutex*, ở processA() và processB() dùng `pthread_mutex_lock(&mutex)` trước khi vào vùng tranh chấp và `pthread_mutex_unlock(&mutex)` sau khi thoát khỏi vùng tranh chấp. Lúc này các số in ra theo đúng thứ tự mong muốn.

II. Bài tập ôn tập

5.6.1. Biến `ans` được tính từ các biến `x1`, `x2`, `x3`, `x4`, `x5`, `x6` như sau:

`w = x1 * x2; (a)`

`v = x3 * x4; (b)`

`y = v * x5; (c)`

`z = v * x6; (d)`

`y = w * y; (e)`

`z = w * z; (f)`

`ans = y + z; (g)`

Giả sử các lệnh từ (a) tới (g) nằm trên các thread chạy song song với nhau. Hãy lập trình mô phỏng và đồng bộ trên C trong hệ điều hành Linux theo thứ tự sau:

- + (c), (d) chỉ được thực hiện sau khi `v` được tính
- + (e) chỉ được thực hiện sau khi `w` và `y` được tính
- + (g) chỉ được thực hiện sau khi `y` và `z` được tính
- **Phương pháp:** dùng sáu semaphore (`sem1` tới `sem6`): `sem1` được nhả khi (a) tính xong; `sem2` được nhả khi (b) tính xong; (c) và (d) chờ `sem2`, tính xong lần lượt nhả `sem3` và `sem4`; (e) chờ `sem1` và `sem3`, tính xong nhả `sem5`, (f) chờ `sem1` và `sem4`, tính xong nhả `sem6`; (g) chờ `sem5` và `6` rồi tính kết quả.

```

32 #include <pthread.h>
31 #include <semaphore.h>
30 #include <stdio.h>
29
28 sem_t sem[6];
27 int ans, v, w, x1, x2, x3, x4, x5, x6, y, z;
26
25 void *
24 processA(void *ignore)
23 {
22     w = x1 * x2;
21     printf("A: w = x1 * x2 = %d * %d = %d\n", x1, x2, w);
20     sem_post(&sem[0]);
19     sem_post(&sem[0]);
18     return NULL;
17 }
16

```

```

34
33 void *
32 processB(void *ignore)
31 {
30 >     v = x3 * x4;
29 >     printf("B: v = x3 * x4 = %d * %d = %d\n", x3, x4, v);
28 >     sem_post(&sem[1]);
27 >     sem_post(&sem[1]);
26 >     return NULL;
25 }
24
23 void *
22 processC(void *ignore)
21 {
20 >     sem_wait(&sem[1]);
19 >     y = v * x5;
18 >     printf("C: y = v * x5 = %d * %d = %d\n", v, x5, y);
17 >     sem_post(&sem[2]);
16 >     return NULL;
15 }
14
13 void *
12 processD(void *ignore)
11 {
10 >     sem_wait(&sem[1]);
9 >     z = v * x6;
8 >     printf("D: z = v * x6 = %d * %d = %d\n", v, x6, z);
7 >     sem_post(&sem[3]);
6 >     return NULL;
5 }
4
17
16 void *
15 processE(void *ignore)
14 {
13 >     sem_wait(&sem[2]);
12 >     sem_wait(&sem[0]);
11 >     y = w * y;
10 >     printf("E: y = w * y = %d * %d = %d\n", y, w, y);
9 >     sem_post(&sem[4]);
8 >     return NULL;
7 }
6

```

```

27 void *
26 processF(void *ignore)
25 {
24 >     sem_wait(&sem[3]);
23 >     sem_wait(&sem[0]);
22 >     z = w * z;
21 >     printf("F: z = w * z = %d * %d = %d\n", z, w, z);
20 >     sem_post(&sem[5]);
19 >     return NULL;
18 }
17
16 void *
15 processG(void *ignore)
14 {
13 >     sem_wait(&sem[4]);
12 >     sem_wait(&sem[5]);
11 >     ans = y + z;
10 >     printf("G: ans = y + z = %d + %d = %d\n", y, z, ans);
9 >     return NULL;
8 }
7
6 void *(*processes[])() = { &processA, &processB, &processC, &processD,
5 >     &processE, &processF, &processG };
6
7 int
8 main(void)
9 {
10 >     pthread_t threads[7];
11 >     scanf("%d %d %d %d %d %d", &x1, &x2, &x3, &x4, &x5, &x6);
12 >     for (int i = 0; i < 6; i++)
13 >         sem_init(&sem[i], 0, 0);
14 >     for (int i = 0; i < 7; i++)
15 >         pthread_create(&threads[i], NULL, processes[i], NULL);
16 >     for (int i = 0; i < 7; i++)
17 >         pthread_join(threads[i], NULL);
18 >     return 0;
19 }

```

- Kết quả chạy:

```
tch:~/build/tmp/os/lab5 % ./561
1 2 3 4 5 6
B: v = x3 * x4 = 3 * 4 = 12
A: w = x1 * x2 = 1 * 2 = 2
D: z = v * x6 = 12 * 6 = 72
F: z = w * z = 144 * 2 = 144
C: y = v * x5 = 12 * 5 = 60
E: y = w * y = 120 * 2 = 120
G: ans = y + z = 120 + 144 = 264
tch:~/build/tmp/os/lab5 %
```

- **Giải thích:** ở hàm main, cho nhập sáu số từ x1 tới x6, khởi tạo sáu semaphore (sem[0] tới sem[5]), và bảy luồng (threads[0] tới threads[6]) chạy bảy hàm từ ProcessA() tới ProcessG(). Hàm main chờ các luồng thực hiện xong rồi thoát.

+ ProcessA() tính xong w rồi tăng sem[0] lên 2 (do có (e) và (f) cùng sử dụng

+ ProcessB() tính xong v rồi tăng sem[1] lên 2 (do có (c) và (d) cùng sử dụng).

+ ProcessC() chờ sem[1] (tức đã có v) rồi tính, sau khi tính xong y , in ra rồi tăng sem[2] lên 1 (tức đã có y).

+ ProcessD() chờ sem[1] (tức đã có v) rồi tính, sau khi tính xong z , in ra rồi tăng sem[3] lên 1 (tức đã có z).

+ ProcessE() chờ sem[0] (tức đã có w) và sem[2] (tức đã có y) rồi tính, sau khi tính xong y *mới*, in ra rồi tăng sem[4] lên 1 (tức đã có y *mới*).

+ ProcessF() chờ sem[0] (tức đã có w) và sem[3] (tức đã có z) rồi tính, sau khi tính xong z *mới*, in ra rồi tăng sem[5] lên 1 (tức đã có z *mới*).

+ ProcessG() chờ sem[4] và sem[5] rồi tính ans rồi in ra.

+ ProcessA, B tăng các semaphore tương ứng lên 2 để đảm bảo các cặp ProcessC và D, E và F có thể được thực thi song song mà không phải chờ nhau nhờ các semaphore tương ứng.